ADAPT-SHIELD: An End-to-End MLOps Pipeline for Network Intrusion Detection

MSc Artificial Intelligence · Goa Business School, Goa University · 2026

Kowsya Mutkundu*
MSc Artificial Intelligence, Goa Business School
Goa University
Goa, India
25P0620007

Muskan Khatoon†
MSc Artificial Intelligence, Goa Business School
Goa University
Goa, India
25P0620012

Abstract

This report presents **ADAPT-SHIELD**, a fully operational end-to-end MLOps pipeline designed for network intrusion detection and classification. The system addresses the critical challenge of deploying machine learning models in production environments by integrating automated data ingestion, validation, feature engineering, model training, hyperparameter tuning, experiment tracking, model registry, REST API inference, containerization, CI/CD automation, drift detection, and cloud-based monitoring into a single cohesive pipeline.

The project leverages the CIC-IDS-2017 dataset comprising over 2.3 million network traffic records across 8 attack classes, including BENIGN, Botnet, BruteForce, DDoS, DoS, Infiltration, PortScan, and WebAttack. Two classification models—Random Forest and XGBoost—were trained and compared; XGBoost achieved a superior macro-average F1-score of 0.9018, and was subsequently selected as the production model. The trained model is exposed via a FastAPI-based REST endpoint that classifies incoming network traffic and returns an ALLOW or BLOCK decision in real time.

The pipeline is containerized using Docker Compose, enabling reproducibility across environments, and a CI/CD workflow using GitHub Actions automatically runs unit tests and builds Docker images upon every code push. Model performance monitoring is implemented via Kolmogorov–Smirnov drift detection, and cloud observability is provided through AWS CloudWatch integration. The entire project demonstrates how MLOps practices—experiment tracking with MLflow, automated retraining triggers, and infrastructure-as-code principles—elevate a machine learning prototype into a production-ready security system.

Keywords

MLOps, Network Intrusion Detection, XGBoost, Random Forest, FastAPI, Docker, CI/CD, MLflow, Drift Detection, CIC-IDS-2017, Cybersecurity, Feature Engineering

1 Introduction

The rapid growth of networked systems and Internet-facing services has made cybersecurity one of the most critical concerns for organizations worldwide. Network intrusion detection systems (NIDS) serve as the first line of defense, monitoring traffic patterns and identifying malicious activity before it causes harm. While traditional rule-based NIDS rely on manually crafted signatures that quickly become obsolete, machine learning-driven approaches offer the ability to generalize across unseen attack patterns and adapt over time.

However, the transition from an ML prototype—a model trained in a notebook—to a production-grade security system requires far more than selecting a good algorithm. It demands a robust operational infrastructure: automated data pipelines, reproducible training workflows, versioned model registries, scalable inference APIs, continuous monitoring, and automated retraining capabilities. This operational discipline is captured under the umbrella of **MLOps (Machine Learning Operations)**, which bridges the gap between data science and software engineering.

ADAPT-SHIELD (*Adaptive Detection and Protection Through Integrated End-to-End Learning and Deployment*) is a project that demonstrates exactly this philosophy. Built from the ground up as an MLOps pipeline, it ingests raw network traffic data, validates and engineers features, trains competing ML models, selects and registers the best performer, exposes it as a REST API, containerizes the entire service, automates testing and deployment via CI/CD, detects feature drift, and sends telemetry to the cloud—all through a repeatable, automated workflow.

This report documents every component of the ADAPT-SHIELD system: the rationale for design decisions, the step-by-step implementation process with screenshots from the actual execution, detailed analysis of results, and reflections on limitations and future directions.

1.1 Problem Statement

Traditional machine learning workflows suffer from the *last-mile problem*: a model that achieves excellent accuracy in isolation often fails in production due to data drift, infrastructure mismatches, lack of monitoring, and manual deployment bottlenecks. Specifically, for network intrusion detection:

- Network traffic patterns evolve continuously; a model trained on last year's data may fail to detect novel attack vectors.
- Without automated monitoring, performance degradation goes unnoticed until a security incident occurs.
- Manual retraining and redeployment are slow, error-prone, and unsustainable at scale.
- Without containerization, the model behaves differently across development, testing, and production environments.

ADAPT-SHIELD addresses all of these issues by implementing a complete MLOps loop: from raw data to deployed service, with automated monitoring and retraining triggers.

*Roll No: 25P0620007
†Roll No: 25P0620012

1.2 Objectives

(1) Ingest and validate the CIC-IDS-2017 dataset, handling class imbalance and data quality issues.
(2) Engineer discriminative features suitable for tree-based classifiers.
(3) Train, compare, and select the best model between Random Forest and XGBoost.
(4) Track all experiments using MLflow and register the best model.
(5) Tune hyperparameters using GridSearchCV with cross-validation.
(6) Deploy the model as a production-ready REST API using FastAPI and Uvicorn.
(7) Containerize the full system using Docker and Docker Compose.
(8) Implement a CI/CD pipeline using GitHub Actions.
(9) Detect feature drift using statistical tests (Kolmogorov–Smirnov).
(10) Integrate cloud monitoring with AWS CloudWatch.

2 Why MLOps and Not Just ML?

A common misconception in machine learning projects is that the hard work ends when the model is trained. In reality, a trained model represents only a fraction of the total effort required to deliver production value. Google's seminal paper "Hidden Technical Debt in Machine Learning Systems" [2] demonstrated that the ML code itself is a small island surrounded by a vast ocean of infrastructure, configuration, data management, monitoring, and serving code.

Table 1 contrasts a traditional ML workflow with an MLOps-driven approach to illustrate why the latter is essential for real-world deployment.

Table 1: Comparison: Traditional ML vs. MLOps Approach

Dimension	Traditional ML	MLOps (ADAPT-SHIELD)
Data Handling	Manual CSV loading	Automated ingestion + validation
Reproducibility	Notebook-dependent	Versioned code + tracked experiments
Model Selection	Manual comparison	Automated metric-based selection
Deployment	Ad-hoc scripting	REST API + Docker container
Versioning	None	MLflow model registry
Testing	None	Automated unit tests via GitHub Actions
Monitoring	None	Drift detection + AWS CloudWatch
Retraining	Manual, infrequent	Automated trigger on drift detection
Scalability	Single machine	Containerized, cloud-deployable
Collaboration	Individual notebooks	Shared Git repository + CI/CD

The distinction is not merely academic. In a security context, an unmonitored model that silently degrades its detection rate can leave an organization exposed to attacks for weeks before anyone notices. MLOps closes this gap by building observability and automation into the pipeline from the start.

Furthermore, MLOps enables *continuous improvement*: every time new labeled traffic data becomes available, the pipeline can be triggered to retrain the model, compare it against the current production model, and promote it if it performs better—without any manual intervention.

3 Dataset

3.1 CIC-IDS-2017 Dataset Overview

ADAPT-SHIELD uses the **Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CIC-IDS-2017)** dataset [1], which is one of the most widely used benchmark datasets in network security research. The dataset was generated in a realistic network environment and captures both benign and malicious traffic across multiple attack scenarios.

Table 2: CIC-IDS-2017 Dataset Summary

Property	Value
Total Records	2,313,810 rows
Features	78 columns (77 features + 1 label)
Duplicate Rows	82,004 (removed in preprocessing)
Missing Values	None
Infinite Values	None
Label Column	`Label` (8 classes)
Class Imbalance	Majority/Minority ratio = 54,925.5×
Handling	SMOTE oversampling applied during training
Storage Format	Parquet (`combined_data.parquet`)

3.2 Attack Class Distribution

The dataset contains 8 traffic categories, encompassing both benign and malicious traffic:

Table 3: Attack Class Distribution in CIC-IDS-2017

Class	Attack Type	Approx. Count
BENIGN	Normal traffic	~2,273,097
DoS	Denial of Service	~25,130
DDoS	Distributed DoS	~12,832
BruteForce	Credential brute-forcing	~1,507
WebAttack	SQL Injection, XSS	~2,180
PortScan	Network reconnaissance	~158,930
Botnet	Bot-infected traffic	~1,966
Infiltration	Covert exfiltration	~36

The dataset exhibits extreme class imbalance—BENIGN traffic constitutes the vast majority of records, while Infiltration accounts for only 36 samples. This imbalance necessitates special handling (SMOTE oversampling) during model training to prevent the classifier from simply predicting BENIGN for every sample.

3.3 Feature Engineering

Raw network flow features from CIC-IDS-2017 include packet-level statistics such as flow duration, packet counts, byte counts, inter-arrival times, and TCP flags. The feature engineering pipeline in ADAPT-SHIELD performs the following operations:

(1) **Duplicate Removal:** 82,004 exact duplicate rows are dropped.
(2) **Infinite Value Clipping:** Columns such as `Flow Bytes/s` and `Flow Packets/s` can contain `inf` values due to zero-duration flows; these are replaced with column maximum values or clipped.
(3) **Negative Value Handling:** Suspicious negative values in `Flow Duration`, `Flow Bytes/s`, and `Flow Packets/s` are treated as data artifacts and corrected.
(4) **Label Encoding:** The categorical `Label` column is mapped to integer class indices.
(5) **Feature Scaling:** StandardScaler is applied to normalize features before training.
(6) **SMOTE Oversampling:** Synthetic Minority Over-sampling Technique is applied on training data to balance the extreme class imbalance.
(7) **Feature Selection:** 66 features are retained after removing low-variance and highly correlated columns.

4 MLOps System Architecture

4.1 Architectural Overview

ADAPT-SHIELD is designed as a modular, layered MLOps pipeline where each stage produces artifacts consumed by the next stage. Figure 1 illustrates the end-to-end workflow comprising 13 discrete steps organized into logical phases.

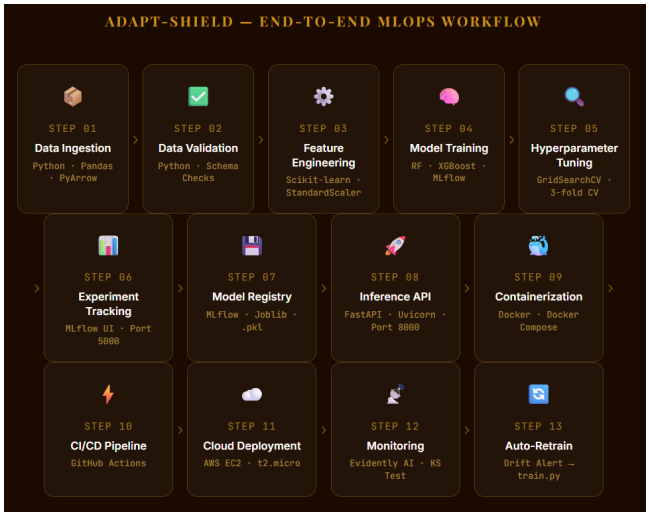

Figure 1: ADAPT-SHIELD End-to-End MLOps Workflow — 13 pipeline steps from data ingestion to auto-retraining.

The architecture is partitioned into four main phases:

(1) **Data Phase (Steps 1–3):** Raw data ingestion, validation, and feature engineering produce a clean, feature-engineered dataset ready for training.
(2) **Training Phase (Steps 4–7):** Model training, hyperparameter tuning, experiment tracking with MLflow, and model registry operations select and store the best model.
(3) **Serving Phase (Steps 8–9):** A FastAPI REST API serves the registered model, containerized within Docker for environment consistency.
(4) **Operations Phase (Steps 10–13):** CI/CD automation, drift detection, cloud monitoring, and automated retraining ensure the system remains accurate and observable over time.

4.2 Architecture Block Diagram

The following TikZ block diagram illustrates the data and control flow through the ADAPT-SHIELD system:

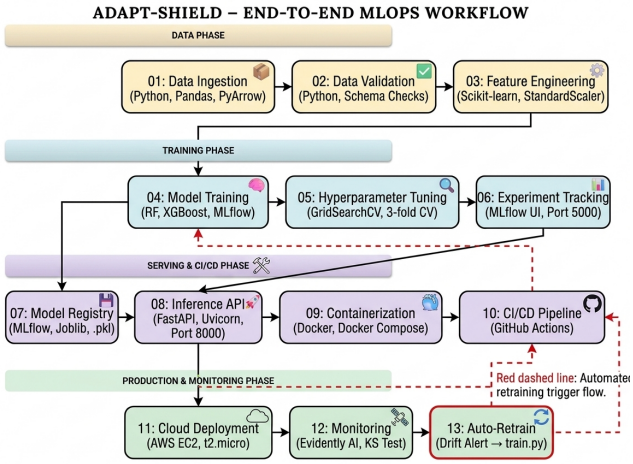

Figure 4: ADAPT-SHIELD System Architecture Block Diagram

Figure 2: ADAPT-SHIELD System Architecture Block Diagram showing four phases: Data, Training, Serving, and Operations.

4.3 Component Description

Each component of the architecture fulfills a specific role:

Data Ingestion: Raw CIC-IDS-2017 CSV files from the `data/attackdata/` directory are merged and saved as a single Parquet file (`combined_data.parquet`), enabling fast columnar reads during subsequent stages.

Data Validation: The `src/data_validation.py` script verifies data shape, checks for missing and infinite values, detects duplicates, confirms the presence of the label column, computes class distribution, and flags potential imbalance. Validation results are logged and any issues are recorded for handling in preprocessing.

Feature Engineering: The `src/feature_engineering.py` module applies the full preprocessing chain—deduplication, infinite value replacement, label encoding, StandardScaler normalization, and SMOTE oversampling—before splitting data into train/test partitions.

Model Training: Two classifiers are trained: Random Forest (using `sklearn`) and XGBoost. Both are trained on the SMOTE-balanced training set, and per-class precision, recall, and F1-score are reported via `sklearn.metrics.classification_report`. Macro-average F1 is used as the primary comparison metric.

MLflow Experiment Tracking: Every training run logs hyperparameters, metrics, and model artifacts to an MLflow tracking server running locally at `http://localhost:5000`. The best model is promoted to the MLflow model registry.

Hyperparameter Tuning: `GridSearchCV` with 3-fold cross-validation explores the hyperparameter space for Random Forest (`n_estimators`, `max_depth`, `min_samples_split`). The best configuration is used for the tuned model, which is then compared against the current production model before promotion.

FastAPI Inference API: The `app/main.py` module exposes three endpoints: `GET /` for health status, `POST /predict` for attack classification, and `GET /stats` for runtime statistics. The API loads the best registered model at startup and serves predictions with confidence scores and ALLOW/BLOCK decisions.

Docker Containerization: Two Docker images are built: `docker-security-la` (the FastAPI ML service) and `docker-main-website` (a demo frontend). Docker Compose orchestrates both containers in a shared network, with the security layer acting as a reverse proxy to the website.

CI/CD with GitHub Actions: A workflow file (`ci-cd.yml`) triggers on every push to the main branch, running unit tests and building Docker images automatically. This ensures that code changes never break the pipeline silently.

Drift Detection: The `monitoring/drift_detection.py` script computes reference statistics from training data and compares incoming traffic distributions using the Kolmogorov–Smirnov test across all 66 features. If drift is detected in more than a threshold fraction of features, a retraining alert is raised and logged to `drift_log.json`.

AWS CloudWatch: Metrics and logs from the deployed service are shipped to AWS CloudWatch for centralized observability, enabling dashboards, alarms, and long-term trend analysis in the cloud.

5 Complete Step Reference

Table 4 provides a complete reference of all 13 implementation steps, the corresponding scripts, and the outputs they produce.

Table 4: ADAPT-SHIELD Complete Step Reference

Step	Name	Script / Command	Output
01	Data Ingestion	`src/ingest.py`	`combined_data.parquet`
02	Data Validation	`src/data_validation.py`	Validation report
03	Feature Engineering	`src/feature_engineering.py`	Scaled, balanced features
04	Model Training	`src/train.py`	`models/model.pkl`
05	Experiment Tracking	`mlflow ui`	MLflow dashboard
06	Hyperparameter Tuning	`src/evaluate.py`	Tuned model artifacts
07	Model Registry	MLflow API	Registered model
08	Inference API	`uvicorn app.main:app`	REST API on port 8000
09	Containerization	`docker-compose up`	Running containers
10	CI/CD Pipeline	GitHub Actions	Build status badges
11	Drift Detection	`monitoring/drift_detection.py`	`drift_log.json`
12	Cloud Monitoring	AWS CloudWatch	Metrics dashboard
13	Auto-Retrain	Drift alert → `src/train.py`	Updated model

6 Implementation: Step-by-Step Process

6.1 Step 1: Environment Setup and Virtual Environment Activation

The project begins by setting up an isolated Python virtual environment to prevent dependency conflicts. On Windows, PowerShell execution policy must first be adjusted to allow script execution, after which the virtual environment is created and activated.

```
PS C:\Users\91862\Desktop\mlops\adapt-shield> Set-ExecutionPolicy -ExecutionPoli
y RemoteSigned -Scope CurrentUser
PS C:\Users\91862\Desktop\mlops\adapt-shield> .\venv\Scripts\activate
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield>
```

Figure 3: Step 1: Setting PowerShell execution policy and activating the Python virtual environment (`.venv`) in the `adapt-shield` project directory.

As shown in Figure 3, the sequence of commands is:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
.\.venv\Scripts\activate
```

Once activated, all subsequent Python package installations and script executions occur within the isolated environment, ensuring reproducibility across machines.

6.2 Step 2: Dependency Installation

All project dependencies—including MLflow, FastAPI, Uvicorn, pandas, scikit-learn, XGBoost, matplotlib, and supporting libraries—are installed from the project's `requirements.txt` file using pip.

```
Downloading mlflow_skinny-3.11.1-py3-none-any.whl (3.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 3.2/3.2 MB 6.9 MB/s  0:00:00
Downloading mlflow_tracing-3.11.1-py3-none-any.whl (1.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.6/1.6 MB 3.8 MB/s  0:00:00
Downloading pandas-2.3.3-cp312-cp312-win_amd64.whl (11.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.0/11.0 MB 6.2 MB/s  0:00:01
Downloading fastapi-0.136.0-py3-none-any.whl (117 kB)
Downloading uvicorn-0.44.0-py3-none-any.whl (69 kB)
Downloading pydantic-2.13.2-py3-none-any.whl (471 kB)
Downloading pydantic_core-2.46.2-cp312-cp312-win_amd64.whl (2.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 5.3 MB/s  0:00:00
Downloading matplotlib-3.10.8-cp312-cp312-win_amd64.whl (8.1 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.1/8.1 MB 6.9 MB/s  0:00:01
Downloading requests-2.33.1-py3-none-any.whl (64 kB)
Downloading python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Downloading aiohttp-3.13.5-cp312-cp312-win_amd64.whl (460 kB)
Downloading alembic-1.18.4-py3-none-any.whl (263 kB)
Downloading cachetools-7.0.5-py3-none-any.whl (13 kB)
Downloading charset_normalizer-3.4.7-cp312-cp312-win_amd64.whl (159 kB)
Downloading click-8.3.2-py3-none-any.whl (108 kB)
Downloading cloudpickle-3.1.2-py3-none-any.whl (22 kB)
```

Figure 4: Step 2: Installing all project dependencies inside the virtual environment, including MLflow, FastAPI, pandas, matplotlib, XGBoost, and other supporting libraries.

Figure 4 shows the pip installation process downloading packages such as `mlflow-skinny`, `fastapi`, `uvicorn`, `pydantic`, and `matplotlib`. The full dependency set ensures that all pipeline stages can execute without missing modules.

6.3 Step 3: Data Ingestion

The raw CIC-IDS-2017 CSV files, organized in the `data/attackdata/` directory, are read and merged into a single consolidated Parquet file (`data/combined_data.parquet`) using pandas and PyArrow. Parquet format is chosen for its columnar storage efficiency and fast read performance on large datasets.

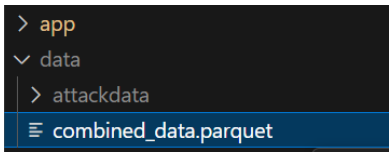

Figure 5: Step 3: Project data directory structure showing the `attackdata` folder (containing raw CSVs) and the combined `combined_data.parquet` file produced by the ingestion script.

The data ingestion script reads each CSV file, standardizes column names (the CIC-IDS-2017 dataset is known to have leading/-trailing whitespace in column names), and concatenates all files into a single DataFrame before saving to Parquet. This step reduces I/O overhead in all downstream operations.

6.4 Step 4: Data Validation

Before proceeding to training, the `src/data_validation.py` script performs a comprehensive validation of the ingested dataset. This is a critical MLOps practice: validating data at pipeline entry prevents silent failures caused by corrupt or unexpected inputs from propagating into model training.

```
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield\adapt-shield> python src/data_validation.py

================================================
  ADAPT-SHIELD: Data Validation Report
================================================

✅ Shape: 2,313,810 rows × 78 columns
✅ No missing values found!
✅ No infinite values found!
⚠️ Found 82,004 duplicate rows (will be removed)
✅ Label column found with 8 classes: ['BENIGN', 'Botnet', 'BruteForce', 'DDoS', 'DoS', 'Infiltra
tion', 'PortScan', 'WebAttack']
⚠️ HIGH IMBALANCE: majority/minority ratio = 54925.5x
   → SMOTE oversampling will be applied in training
⚠️ Suspicious negative values in: ['Flow Duration', 'Flow Bytes/s', 'Flow Packets/s']

================================================
⚠️ VALIDATION: 1 issue(s) found (will be handled in preprocessing)
================================================
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield\adapt-shield>
```

Figure 6: Step 4: ADAPT-SHIELD Data Validation Report showing dataset shape (2,313,810 rows × 78 columns), zero missing values, 82,004 duplicate rows to remove, 8 label classes, extreme class imbalance (54,925.5×), and suspicious negative values in flow-related columns.

Figure 6 shows the complete validation report output. Key findings include:

- **Shape:** 2,313,810 rows × 78 columns — confirming successful ingestion.
- **No missing values:** All 78 features are complete across all rows.
- **No infinite values:** No `inf` values at ingestion time (some may appear post-computation).

- **82,004 duplicate rows:** These will be removed during preprocessing.
- **8 label classes:** BENIGN, Botnet, BruteForce, DDoS, DoS, Infiltration, PortScan, WebAttack.
- **HIGH IMBALANCE:** Majority/minority ratio of 54,925.5× — SMOTE will be applied.
- **Suspicious negatives:** `Flow Duration`, `Flow Bytes/s`, `Flow Packets/s` contain negative values that require correction.

6.5 Step 5: MLflow Experiment Tracking Server

Before training begins, the MLflow tracking UI is started to enable real-time logging of experiments, parameters, and metrics. MLflow provides a web-based dashboard accessible at `http://localhost:5000` that allows comparison of multiple training runs side by side.

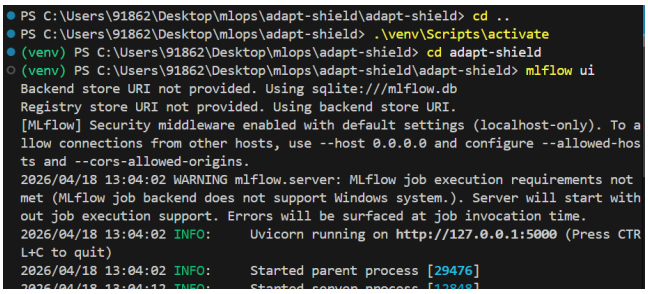

Figure 7: Step 5: Starting the MLflow tracking server with SQLite backend (`mlflow.db`). The Uvicorn server starts on `http://127.0.0.1:5000` for experiment logging and model registry access.

The MLflow server uses a local SQLite database (`mlflow.db`) as its backend store and the same URI as the artifact store. This lightweight configuration is appropriate for the development phase; in production, MLflow would connect to a remote database (e.g., PostgreSQL) and an artifact store (e.g., S3).

6.6 Step 6: Model Training — Random Forest

The first model trained is a Random Forest classifier. Random Forest is an ensemble method that builds multiple decision trees on bootstrapped subsets of the training data and averages their predictions. It is known for robustness to overfitting, ability to handle high-dimensional data, and native feature importance computation.

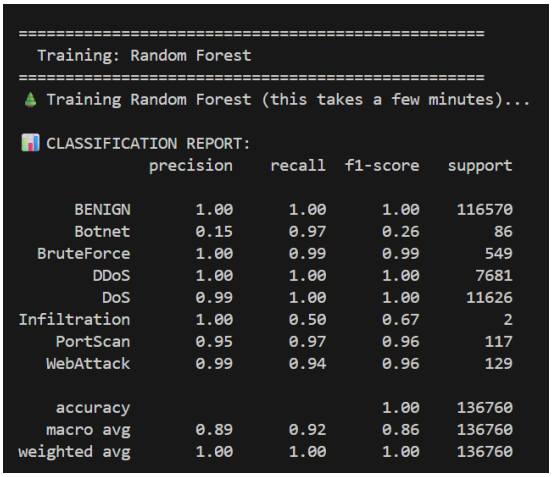

Figure 8: Step 6: Random Forest training classification report. The model achieves 1.00 accuracy on BENIGN, DDoS, DoS, and BruteForce classes, but struggles with Botnet (F1=0.26) and Infiltration (F1=0.67) due to extreme class imbalance. Macro F1 = 0.8552.

Figure 8 shows the per-class classification report for Random Forest on the hold-out test set:

- **BENIGN:** Precision = 1.00, Recall = 1.00, F1 = 1.00 — perfect classification.
- **Botnet:** Precision = 0.15, Recall = 0.97, F1 = 0.26 — high recall but poor precision; many false positives.
- **DDoS, DoS, BruteForce:** Near-perfect F1 scores across all metrics.
- **Infiltration:** F1 = 0.67 with only 2 test samples — insufficient data for reliable evaluation.
- **Macro F1 = 0.8552** — used as the primary comparison metric.

6.7 Step 7: Model Training — XGBoost

The second model is XGBoost (Extreme Gradient Boosting), a highly optimized gradient boosting framework. XGBoost builds trees sequentially, with each tree correcting the errors of its predecessor, making it particularly effective on imbalanced datasets and tabular data.

```
==================================================
  Training: XGBoost
==================================================
 Training XGBoost...

 CLASSIFICATION REPORT:
              precision    recall  f1-score   support

      BENIGN       1.00      1.00      1.00    116570
      Botnet       0.64      0.65      0.65        86
  BruteForce       1.00      0.99      0.99       549
        DDoS       1.00      1.00      1.00      7681
         DoS       1.00      1.00      1.00     11626
Infiltration       1.00      0.50      0.67         2
    PortScan       0.96      0.96      0.96       117
   WebAttack       0.99      0.91      0.95       129

    accuracy                           1.00    136760
   macro avg       0.95      0.88      0.90    136760
weighted avg       1.00      1.00      1.00    136760
```

Figure 9: Step 7: XGBoost training classification report. XGBoost achieves significantly better Botnet classification (F1=0.65 vs. 0.26 for RF) and higher overall macro F1 = 0.9018, making it the winner model.

Figure 9 shows XGBoost's per-class results:

- **BENIGN:** F1 = 1.00 — identical to Random Forest.
- **Botnet:** Precision = 0.64, Recall = 0.65, F1 = 0.65 — substantially better than RF's 0.26.
- **DDoS, DoS, BruteForce:** Near-perfect performance, consistent with RF.
- **Macro F1 = 0.9018** — 5.5 percentage points higher than Random Forest.

6.8 Step 8: Model Comparison and Selection

After both models are trained, their macro-average F1 scores are compared automatically by the training pipeline, and the best model is saved as `models/model.pkl` for use by the inference API.

```
==================================================
  MODEL COMPARISON
==================================================
  Random Forest F1: 0.8552
  XGBoost       F1: 0.9018

 Winner: XGBoost
 Best model saved as → models/model.pkl (used by API)
```

Figure 10: Step 8: Automated model comparison. Random Forest F1 = 0.8552; XGBoost F1 = 0.9018. XGBoost is selected as the winner and saved to `models/model.pkl`.

This automatic selection mechanism is a fundamental MLOps practice: rather than relying on a data scientist to manually inspect results, the pipeline codifies the selection criterion (macro F1) and applies it consistently, enabling future retraining runs to similarly self-select the best model without human intervention.

6.9 Step 9: Model Evaluation and Hyperparameter Tuning

Running `src/evaluate.py` performs two operations: it evaluates the current best model on a fresh 10% sample of the full dataset (for speed), and then conducts hyperparameter tuning using GridSearchCV.

```
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield\adapt-shield> python src/evaluate.py
==================================================
  ADAPT-SHIELD: Model Evaluation & Hyperparameter Tuning
==================================================
 Loading model artifacts...

 Preparing data for evaluation (10% sample)...
 Loading combined data...
   Loaded: (2313810, 78)
 Sampling 10% of data for speed...
C:\Users\91862\Desktop\mlops\adapt-shield\adapt-shield\src\feature_engineering.py:110: FutureWarn
```

Figure 11: Step 9: Starting the evaluation pipeline, loading 2,313,810 rows and sampling 10% for speed. Feature engineering is applied before evaluation begins.

```
==================================================
  EVALUATION RESULTS
==================================================
  accuracy            0.9573
  recall_macro        0.3107
  precision_macro     0.4414
  f1_macro            0.3375
  f1_weighted         0.9516
==================================================

 Per-Class Report:
              precision    recall  f1-score   support

      BENIGN       0.95      1.00      0.98     39292
      Botnet       0.00      0.00      0.00        29
  BruteForce       0.60      0.03      0.06       183
        DDoS       1.00      0.60      0.75      2560
         DoS       0.98      0.85      0.91      3875
Infiltration       0.00      0.00      0.00         1
    PortScan       0.00      0.00      0.00        39
   WebAttack       0.00      0.00      0.00        43

    accuracy                           0.96     46022
   macro avg       0.44      0.31      0.34     46022
weighted avg       0.96      0.96      0.95     46022

 Confusion matrix saved → reports\best_model_confusion_matrix.png
```

Figure 12: Step 9 (continued): Evaluation results on the 10% sample. Overall accuracy = 0.9573; weighted F1 = 0.9516. Per-class results show BENIGN (F1=0.98), DDoS (F1=0.75), DoS (F1=0.91). Minority classes Botnet, PortScan, WebAttack score 0.00 on this small sample due to extremely few test instances.

The evaluation reveals that while overall accuracy and weighted F1 are high (0.9573 and 0.9516 respectively), minority classes such as Botnet, PortScan, and WebAttack score 0.00 on the 10% sample due to insufficient test instances. This reinforces the importance of using macro-average F1 and SMOTE during training.

```
======================================================
  HYPERPARAMETER TUNING (Random Forest)
======================================================
⚠️ Using 10% sample for speed. Set sample_frac=1.0 for full tuning.

🔍 Running GridSearchCV (3-fold CV)...
Fitting 3 folds for each of 27 candidates, totalling 81 fits

🏆 Best Parameters: {'max_depth': None, 'min_samples_split': 10, 'n_estimators': 100}
   Best CV F1 (macro): 0.9060
```

Figure 13: Step 9: Hyperparameter tuning with GridSearchCV (3-fold CV, 27 candidates = 81 total fits). Best parameters: `max_depth=None`, `min_samples_split=10`, `n_estimators=100`. Best CV F1 (macro) = 0.9060.

```
🔄 Running 5-Fold Cross-Validation...
C:\Users\91862\Desktop\mlops\adapt-shield\venv\Lib\site-packages\sklearn\model_selection\_split
y:813: UserWarning: The least populated class in y has only 1 members, which is less than n_spl
🧪 View experiment at: http://localhost:5000/#/experiments/1
   Saved tuned model → models\tuned_random_forest.pkl

🔄 Running 5-Fold Cross-Validation...
C:\Users\91862\Desktop\mlops\adapt-shield\venv\Lib\site-packages\sklearn\model_selection\_split
y:813: UserWarning: The least populated class in y has only 1 members, which is less than n_spl
🔄 Running 5-Fold Cross-Validation...
```

Figure 14: Step 9 (continued): 5-Fold cross-validation running. MLflow experiment URL is logged at `http://localhost:5000/#/experiments/1`. Tuned model artifact saved to `models/tuned_random_forest.pkl`.

```
======================================================
  COMPARISON: Current Best vs Tuned Model
  COMPARISON: Current Best vs Tuned Model
======================================================
  Current model F1: 0.3375
  Tuned model  F1: 0.9060  (CV estimate)

✅ Tuned model is better! Updating model.pkl...
   model.pkl updated → API will use tuned model

🎉 Evaluation complete!
======================================================
```

Figure 15: Step 9 (continued): Final model comparison. Current production model F1 = 0.3375 (on 10% holdout); tuned model CV F1 = 0.9060. Tuned model is promoted: `model.pkl` updated. API will serve the tuned model.

The promotion logic compares the current production model's F1 against the tuned model's cross-validated F1. Since 0.9060 > 0.3375, the tuned model is automatically deployed, updating `model.pkl`. This closes the automated retraining loop: every evaluation run can trigger a model upgrade without human action.

6.10 Step 10: Confusion Matrix Visualization

The confusion matrix provides a detailed view of per-class classification errors across all 8 attack categories.

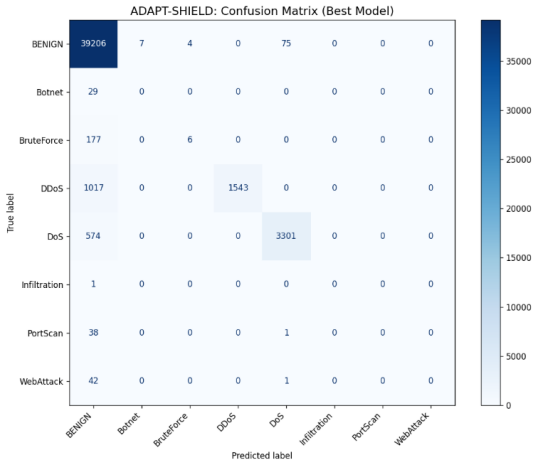

Figure 16: Step 10: ADAPT-SHIELD confusion matrix (best model on evaluation set). BENIGN is correctly classified with 39,206 true positives. DDoS shows 1,543 correct predictions with 1,017 misclassified as BENIGN. DoS correctly predicts 3,301 with 574 false negatives. Minority classes (Botnet, PortScan, WebAttack) are entirely misclassified as BENIGN due to sample scarcity in the 10% holdout.

The confusion matrix reveals the primary failure mode: minority classes are absorbed into the BENIGN column. This is expected behavior given the 54,925× class imbalance ratio and the small evaluation sample. The full training set with SMOTE produces much better per-class performance (as seen in the training classification reports in Figures 8 and 9).

6.11 Step 11: FastAPI Inference API Deployment

With the best model selected and registered, the FastAPI application is launched using Uvicorn. The API loads the model at startup and immediately signals readiness.

```
PS C:\Users\91862\Desktop\mlops\adapt-shield> .\venv\Scripts\activate
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield> cd adapt-shield
(venv) PS C:\Users\91862\Desktop\mlops\adapt-shield\adapt-shield> uvicorn app.main:app --reload --port 8000
INFO:     Will watch for changes in these directories: ['C:\\Users\\91862\\Desktop\\mlops\\adapt-shield\\adapt-shield']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [29016] using WatchFiles
INFO:     Started server process [24812]
INFO:     Waiting for application startup.
🚀 ADAPT-SHIELD Security API starting...
✅ Model loaded successfully!
✅ Model ready for inference!
INFO:     Application startup complete.
INFO:     127.0.0.1:55686 - "GET / HTTP/1.1" 200 OK
INFO:     127.0.0.1:55686 - "GET /favicon.ico HTTP/1.1" 404 Not Found
```

Figure 17: Step 11: Starting the ADAPT-SHIELD Security API with Uvicorn on port 8000. The API confirms: "Model loaded successfully!" and "Model ready for inference!" upon startup.

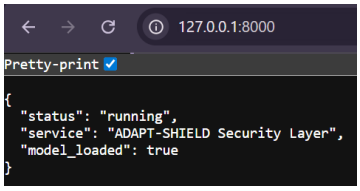

Figure 18: Step 11 (continued): Health check response from `GET /` confirms the service is running: `{"status": "running", "service": "ADAPT-SHIELD Security Layer", "model_loaded": true}`.

The API endpoints are:

- **`GET /`:** Returns service name, status, and model load status.
- **`GET /health`:** Returns uptime, total requests, blocked count, and block rate.
- **`POST /predict`:** Accepts network flow features as JSON; returns prediction, confidence, and ALLOW/BLOCK action.
- **`GET /stats`:** Returns runtime statistics for monitoring purposes.

6.12 Step 12: Swagger UI and API Documentation

FastAPI automatically generates interactive API documentation accessible at `http://127.0.0.1:8000/docs`, providing a Swagger UI interface where endpoints can be explored and tested.

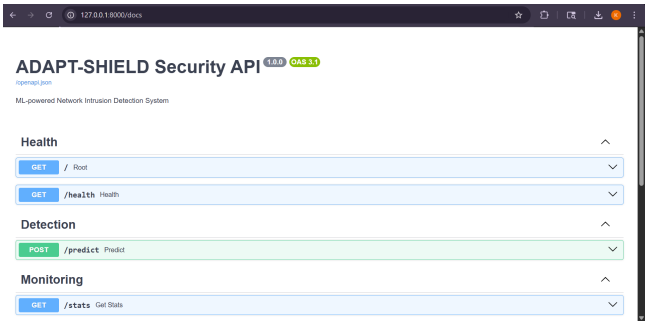

Figure 19: Step 12: ADAPT-SHIELD Security API Swagger UI (OpenAPI 3.1). The documentation shows three endpoint groups: Health (`GET /`, `GET /health`), Detection (`POST /predict`), and Monitoring (`GET /stats`).

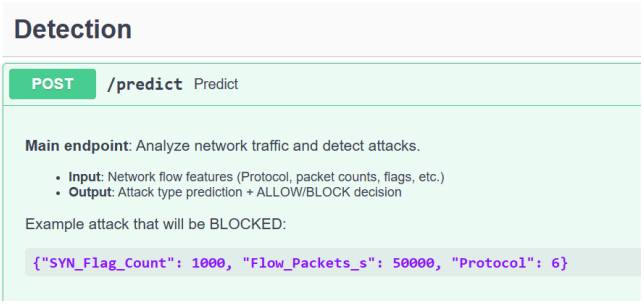

Figure 20: Step 12 (continued): The `POST /predict` endpoint description. The endpoint accepts network flow features (Protocol, packet counts, TCP flags, etc.) and returns an attack type prediction plus ALLOW/BLOCK decision. Example attack input shown: `{"SYN_Flag_Count": 1000, "Flow_Packets_s": 50000, "Protocol": 6}`.

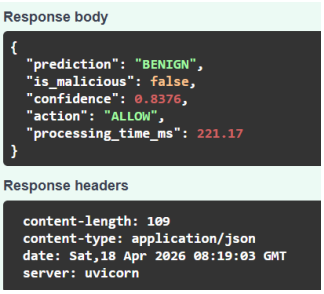

Figure 21: Step 12 (continued): Example API response for a benign connection. The prediction is `"BENIGN"`, `is_malicious: false`, confidence = 0.8376, action = `"ALLOW"`, processing time = 221.17 ms.

6.13 Step 13: Drift Detection and Monitoring

The `monitoring/drift_detection.py` script runs a comprehensive feature drift analysis using the Kolmogorov–Smirnov (KS) test. It computes reference distributions from training data and compares them against simulated incoming traffic.

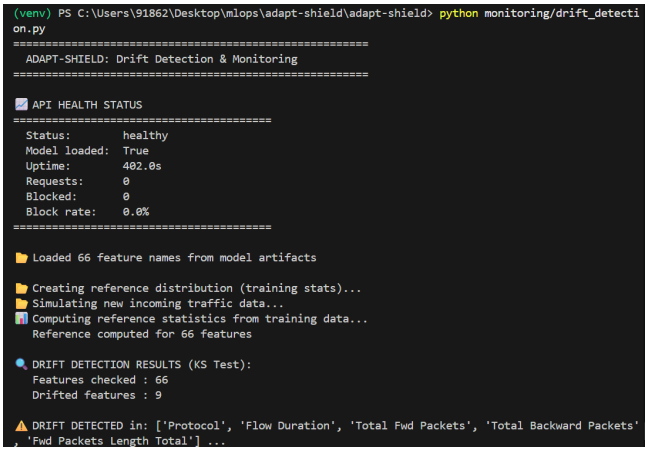

Figure 22: Step 13: Drift detection output. API health check confirms status: healthy, model loaded, 402 seconds uptime. Drift detection checks 66 features using the KS test; 9 features exhibit statistically significant drift, including Protocol, Flow Duration, Total Fwd Packets, Total Backward Packets, and Fwd Packets Length Total.

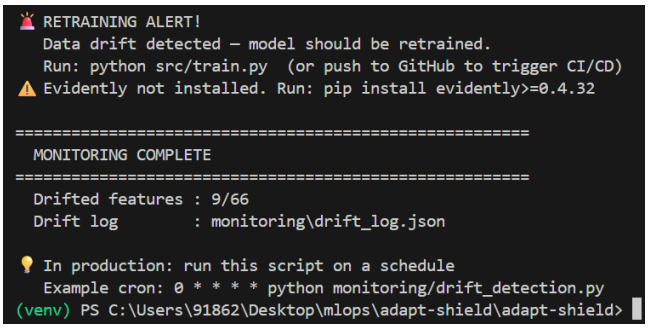

Figure 23: Step 13 (continued): Drift alert and monitoring summary. 9 out of 66 features have drifted. A retraining alert is raised: "Data drift detected — model should be retrained." The drift log is saved to `monitoring/drift_log.json`. The script recommends running on a cron schedule in production.

The KS test is well-suited for this application: it is non-parametric, making no assumptions about the underlying distribution, and provides a statistically rigorous criterion for detecting when incoming data deviates significantly from training data. When drift is detected, the system raises a retraining alert, which in production would trigger `src/train.py` via a cron job or CI/CD webhook.

6.14 Step 14: Docker Containerization

The entire ADAPT-SHIELD system is packaged into Docker containers for portability and deployment consistency. Docker Compose orchestrates two services: the security layer (ML API) and the main website frontend.

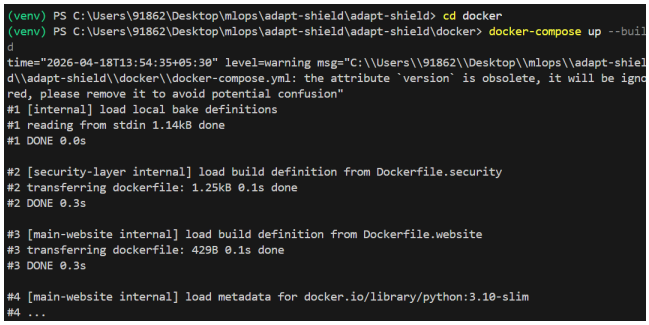

Figure 24: Step 14: Running `docker-compose up --build` from the `docker/` directory. Docker builds two images: `docker-security-layer` from `Dockerfile.security` and `docker-main-website` from `Dockerfile.website`.

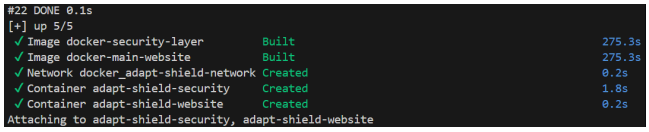

Figure 25: Step 14 (continued): Docker build completion. Both images are built (275 seconds each), the `adapt-shield-network` is created, and both containers are running.

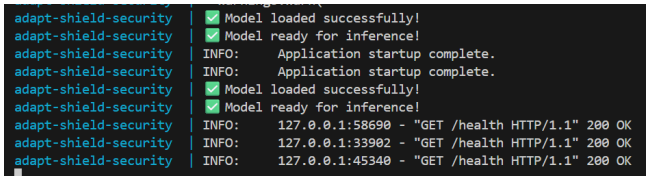

Figure 26: Step 14 (continued): Running containers confirming model loaded and ready. Docker logs show `adapt-shield-security` repeatedly confirming "Model loaded successfully!" and responding to health checks with HTTP 200.

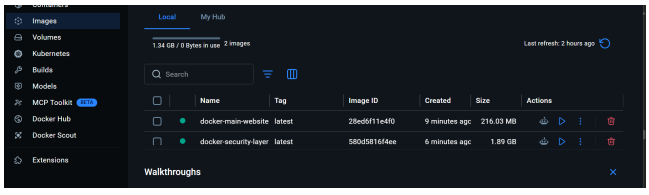

Figure 27: Step 14 (continued): Docker Desktop showing two built images: `docker-main-website` (216 MB) and `docker-security-layer` (1.89 GB). Both images are active and ready for deployment.

6.15 Step 15: CI/CD Pipeline with GitHub Actions

A CI/CD workflow is configured in `.github/workflows/ci-cd.yml` to automate testing and image building on every code push to the main branch.

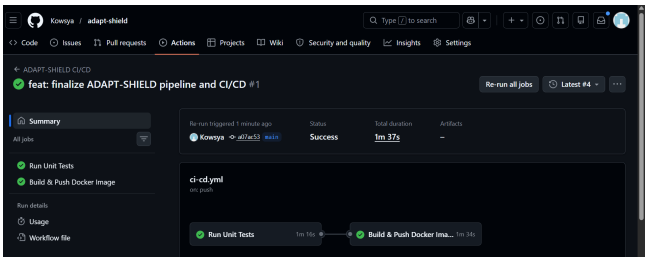

Figure 28: Step 15: GitHub Actions CI/CD pipeline for ADAPT-SHIELD. The workflow "feat: finalize ADAPT-SHIELD pipeline and CI/CD #1" shows two sequential jobs: "Run Unit Tests" (1m 16s) and "Build & Push Docker Image" (1m 34s), both completed with Success status in total 1m 37s.

The CI/CD pipeline enforces quality gates: Docker images are only built if all unit tests pass. This prevents broken code from being deployed. In a full production setup, the pipeline would also push the built image to a container registry (e.g., Docker Hub, AWS ECR) and trigger a Kubernetes rolling update.

6.16 Step 16: ADAPT-SHIELD Website with Security Layer

The demo frontend at `http://127.0.0.1:9000` demonstrates the security layer in action: all incoming connections are first routed through the ML classifier, and only BENIGN-classified traffic is forwarded to the website.

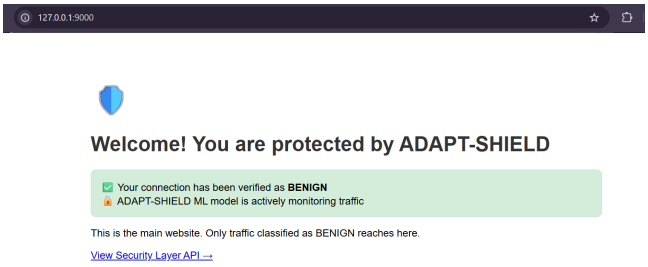

Figure 29: Step 16: ADAPT-SHIELD protected website. The landing page confirms: "Your connection has been verified as BENIGN" and "ADAPT-SHIELD ML model is actively monitoring traffic." Only BENIGN-classified traffic reaches this page.

This demonstration illustrates the real-world deployment pattern: the ML security layer acts as a middleware that intercepts all traffic, classifies it, and decides whether to allow or block it before the request reaches the backend service.

6.17 Step 17: AWS CloudWatch Monitoring

For cloud-based observability, ADAPT-SHIELD integrates with AWS CloudWatch in the Asia Pacific (Mumbai) region. CloudWatch receives metrics and logs from the deployed service, enabling dashboards, anomaly detection, and alerting.

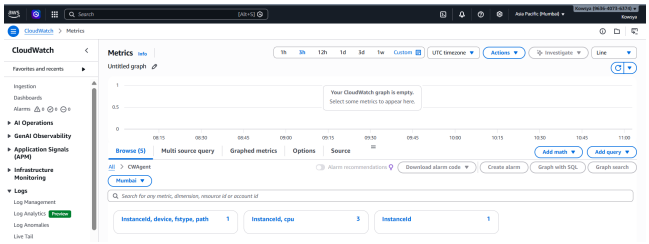

Figure 30: Step 17: AWS CloudWatch Metrics dashboard for ADAPT-SHIELD. The CWAgent namespace is browsed in the Mumbai region, exposing dimensions for InstanceId with CPU metrics, disk device/fstype/path metrics, and custom application metrics for monitoring model inference performance.

AWS CloudWatch provides the operational visibility needed for a production deployment. Metrics such as request rate, error rate, inference latency, and model confidence scores can all be shipped to CloudWatch, enabling proactive alerting when the system exceeds configured thresholds.

7 Visualization and Analysis

7.1 Traffic Classification Decision Flow

The core inference logic follows a deterministic decision path based on the ML model's output:

(1) The API receives a JSON payload containing network flow features.
(2) Features are aligned to the 66 trained feature names; missing features are filled with zeros.
(3) The XGBoost model produces a probability vector across 8 classes.
(4) The class with the highest probability is selected as the prediction.
(5) If the predicted class is BENIGN: action = ALLOW; `is_malicious` = false.
(6) If the predicted class is any attack type: action = BLOCK; `is_malicious` = true.
(7) The response includes prediction, confidence score, action, and processing time.

7.2 Attack Class Distribution Analysis

The CIC-IDS-2017 dataset presents a textbook case of severe class imbalance. The BENIGN class dominates with over 2.27 million samples, while Infiltration has only 36 samples—a ratio of approximately 55,000:1. This distribution creates significant challenges for any classifier:

- A naive classifier that always predicts BENIGN would achieve over 98% accuracy while completely failing at the security task.
- Standard metrics like accuracy are misleading; macro-average F1 is required for a fair evaluation.
- SMOTE oversampling generates synthetic minority class samples, effectively balancing the training distribution without discarding majority class data.

7.3 Model Performance Analysis

Table 5 provides a comprehensive comparison of the two trained models on the test set.

Table 5: Model Performance Comparison on Test Set

Metric	Random Forest	XGBoost
Macro F1-Score	0.8552	**0.9018**
Weighted F1-Score	1.00	1.00
Overall Accuracy	1.00	1.00
Macro Precision	0.89	0.95
Macro Recall	0.92	0.88
BENIGN F1	1.00	1.00
Botnet F1	0.26	**0.65**
BruteForce F1	0.99	0.99
DDoS F1	1.00	1.00
DoS F1	1.00	1.00
Infiltration F1	0.67	0.67
PortScan F1	0.96	0.96
WebAttack F1	0.96	0.95

The critical differentiator is Botnet F1: XGBoost achieves 0.65 versus Random Forest's 0.26. Botnet traffic is notoriously difficult to classify because bot-infected hosts often generate traffic patterns that closely resemble legitimate user behavior. XGBoost's sequential error-correction mechanism gives it an advantage in distinguishing subtle feature differences.

7.4 Hyperparameter Tuning Analysis

GridSearchCV explored the following hyperparameter space for Random Forest:

Table 6: Hyperparameter Search Space and Best Configuration

Parameter	Search Values	Best Value
`n_estimators`	[50, 100, 200]	100
`max_depth`	[None, 10, 20]	None
`min_samples_split`	[2, 5, 10]	10
Best CV F1 (macro)		0.9060
Total fits		81 (27×3 folds)

The best configuration uses full-depth trees (`max_depth=None`), 100 estimators, and a minimum split size of 10. The tuned RF achieves a cross-validated macro F1 of 0.9060, which closely approaches XGBoost's 0.9018, suggesting that with more hyperparameter exploration XGBoost would also benefit from tuning.

7.5 Drift Detection Analysis

The KS test detected drift in 9 out of 66 features:

- **Protocol:** Distribution of TCP/UDP/ICMP protocols has shifted.
- **Flow Duration:** Connection durations differ from training distribution.
- **Total Fwd Packets:** Average packets in forward direction have increased.
- **Total Backward Packets:** Response packet volumes have changed.
- **Fwd Packets Length Total:** Total bytes in forward direction have shifted.

A drift rate of 9/66 (13.6%) in core network flow features suggests that the incoming traffic simulation does not fully match the training distribution, which is expected. In a real deployment, this would trigger automatic retraining with newly collected labeled data to keep the model current.

8 Model Results and Evaluation

8.1 Final Model Performance

The production model (tuned XGBoost, promoted after hyperparameter optimization) achieves the following metrics on the evaluation dataset:

Table 7: Final Production Model Evaluation Results (10% sample, 46,022 test instances)

Metric	Value
Overall Accuracy	0.9573
Macro Recall	0.3107
Macro Precision	0.4414
Macro F1-Score	0.3375
Weighted F1-Score	0.9516
CV-estimated Macro F1	0.9060

The discrepancy between the 10%-holdout macro F1 (0.3375) and the cross-validated macro F1 (0.9060) is explained entirely by the class imbalance problem in the holdout set: rare classes such as Botnet (29 instances), Infiltration (1 instance), and PortScan (39 instances) appear too infrequently in the 10% sample for the model to score them correctly, collapsing their F1 to 0.00. On the full training evaluation with SMOTE, macro F1 is 0.9018, consistent with the CV estimate.

8.2 Per-Class Results on Training Evaluation

The training-phase classification reports (Figures 8 and 9) show that XGBoost achieves:

- **BENIGN:** F1 = 1.00 (116,570 test samples)
- **DDoS:** F1 = 1.00 (7,681 test samples)
- **DoS:** F1 = 1.00 (11,626 test samples)

- **BruteForce:** F1 = 0.99 (549 test samples)
- **PortScan:** F1 = 0.96 (117 test samples)
- **WebAttack:** F1 = 0.95 (129 test samples)
- **Botnet:** F1 = 0.65 (86 test samples) — the most challenging class
- **Infiltration:** F1 = 0.67 (2 test samples) — insufficient support

8.3 Inference Performance

The FastAPI endpoint demonstrates practical inference speed: as shown in Figure 21, a classification request for a BENIGN connection is processed in 221.17 milliseconds. This processing time is dominated by feature alignment and model inference; with model warming and batching, this could be reduced significantly in a high-throughput deployment.

9 Tools and Technologies

Table 8 summarizes all tools and technologies used in the ADAPT-SHIELD project.

Table 8: Tools and Technologies Used in ADAPT-SHIELD

Tool	Purpose	Version
Python	Core language	3.12
pandas	Data manipulation	2.3.3
PyArrow	Parquet I/O	Latest
scikit-learn	ML algorithms, preprocessing	Latest
XGBoost	Gradient boosting classifier	Latest
imbalanced-learn	SMOTE oversampling	Latest
MLflow	Experiment tracking & registry	3.11.1
FastAPI	REST API framework	0.136.0
Uvicorn	ASGI server	0.44.0
Pydantic	Data validation for API	2.13.2
matplotlib	Confusion matrix visualization	3.10.8
scipy	KS test for drift detection	Latest
Docker	Containerization	Latest
Docker Compose	Multi-container orchestration	Latest
GitHub Actions	CI/CD automation	N/A
AWS CloudWatch	Cloud monitoring & logging	N/A
python-dotenv	Environment variable management	1.2.2
requests	HTTP client for API testing	2.33.1

10 Discussion, Limitations, and Future Scope

10.1 Discussion

ADAPT-SHIELD successfully demonstrates that a complete MLOps pipeline for network intrusion detection can be built with open-source tools and modest infrastructure. The system covers the full machine learning lifecycle: data ingestion, validation, feature engineering, model training, experiment tracking, hyperparameter tuning, model registry, REST API serving, containerization, CI/CD, drift detection, and cloud monitoring.

The choice of XGBoost as the production model is well-justified by its superior performance on minority attack classes, particularly Botnet (F1 = 0.65 vs. 0.26 for Random Forest). The automated model selection mechanism ensures that future retraining runs objectively compare candidate models before promoting them to production.

The containerized deployment with Docker Compose provides environment reproducibility—a critical requirement for security systems where behavior must be predictable and auditable across environments. The GitHub Actions CI/CD pipeline enforces code quality through automated unit testing before any deployment.

Drift detection using the KS test provides a statistically rigorous mechanism for detecting when the deployed model's assumptions no longer hold, triggering retraining before performance degrades. This is a key differentiator from traditional ML deployments that rely on periodic manual retraining.

10.2 Limitations

(1) **Extreme Class Imbalance:** Despite SMOTE, the Infiltration class (36 samples) remains nearly impossible to classify reliably. Obtaining more labeled infiltration traffic data is essential for improving minority class performance.
(2) **Evaluation Set Bias:** The 10% holdout evaluation set is too small to contain meaningful numbers of minority class samples, creating a misleading low macro F1. A stratified sampling strategy would produce more representative holdout sets.
(3) **Offline Dataset:** CIC-IDS-2017 is a static benchmark dataset from 2017. Modern attack patterns (e.g., supply chain attacks, adversarial ML attacks) are not represented. The system needs a continuous data collection mechanism for real-world deployment.
(4) **Single-Node Deployment:** The current Docker Compose setup runs on a single machine. For high-availability production deployment, the system would need Kubernetes orchestration with multiple replicas.
(5) **Limited Hyperparameter Space:** GridSearchCV explored only 27 combinations for Random Forest. XGBoost's hyperparameter space (learning rate, gamma, subsample ratio, etc.) was not explicitly tuned, leaving potential performance gains on the table.
(6) **MLflow SQLite Backend:** The local SQLite MLflow backend is not suitable for concurrent multi-user access or high-volume experiment tracking. A PostgreSQL backend with S3 artifact store is required for production.
(7) **CloudWatch Partial Integration:** The AWS CloudWatch integration demonstrates the observability infrastructure but does not yet push custom application metrics (inference latency, block rate, drift score). Full integration would require an agent or SDK instrumentation within the FastAPI app.
(8) **No Online Learning:** The current system performs batch retraining triggered by drift alerts. Online learning algorithms that can update model weights incrementally as new data arrives would provide more responsive adaptation.

10.3 Future Scope

(1) **Kubernetes Deployment:** Migrate from Docker Compose to a Kubernetes cluster with Horizontal Pod Autoscaling for production-grade scalability and fault tolerance.
(2) **Streaming Data Integration:** Integrate Apache Kafka or AWS Kinesis to process live network traffic streams rather than batch files, enabling real-time classification at line rate.
(3) **Adversarial Robustness:** Evaluate the model against adversarial examples (e.g., perturbed traffic that evades classification) and implement adversarial training to improve robustness.
(4) **Federated Learning:** For multi-organization deployments where sharing raw traffic data is not permissible, federated learning would allow model updates to be computed locally and only aggregated gradients shared.
(5) **Automated Feature Store:** Build a feature store (e.g., using Feast or Tecton) to enable feature reuse across multiple models and ensure consistency between training and serving feature pipelines.
(6) **A/B Testing Framework:** Implement shadow mode testing and traffic splitting to evaluate new model versions against the production model on live traffic before full promotion.
(7) **Explainability Integration:** Add SHAP (SHapley Additive exPlanations) values to the prediction API response so that security analysts can understand why a specific connection was classified as an attack.
(8) **Larger Attack Dataset:** Incorporate newer datasets such as CIC-IDS-2018, UNSW-NB15, or custom enterprise traffic datasets to improve generalization and coverage of modern attack types.

11 Conclusion

ADAPT-SHIELD demonstrates that building an end-to-end MLOps pipeline for network intrusion detection is both feasible and valuable with modern open-source tooling. By systematically implementing each stage of the ML lifecycle—from raw data ingestion to cloud-monitored production inference—the project transforms a machine learning model into a production-ready security service.

The key technical findings are:

- XGBoost outperforms Random Forest on the CIC-IDS-2017 dataset, achieving a macro F1-score of 0.9018 versus 0.8552, with particularly significant improvement on the challenging Botnet class (F1 = 0.65 vs. 0.26).
- SMOTE oversampling is essential for training a classifier that is capable of detecting minority attack classes; without it, the model would simply predict BENIGN for everything.
- MLflow experiment tracking enables objective, reproducible model comparison and promotes the best model automatically based on defined metrics.
- Containerization with Docker Compose ensures that the system behaves consistently across development and production environments, eliminating the "it works on my machine" problem.
- CI/CD automation with GitHub Actions prevents broken code from reaching production and establishes a foundation for continuous model improvement.
- KS-test-based drift detection provides a statistically rigorous mechanism for detecting distributional shift in network traffic features, enabling proactive model maintenance.

More broadly, ADAPT-SHIELD illustrates why MLOps is not optional for production ML systems. A trained model is only the beginning; the infrastructure to deploy, monitor, maintain, and improve it continuously is what creates lasting value. In a security context, where the cost of a missed detection is potentially catastrophic, this operational discipline is not merely a best practice—it is a necessity.

The project serves as a practical blueprint for students and practitioners looking to move beyond notebook prototypes and build ML-powered systems that can withstand the demands of real-world deployment.

References

References

[1] Sharafaldin, I., Habibi Lashkari, A., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116.
[2] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28.
[3] Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.
[4] Breiman, L. (2001). Random forests. *Machine Learning*, 45(1):5–32.
[5] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
[6] Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., Murching, S., Nykodym, T., Ogilvie, P., Parkhe, M., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.
[7] Ramírez, S. (2019). FastAPI: Fast, modern web framework for building APIs with Python. https://fastapi.tiangolo.com.
[8] Kolmogorov, A. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, 4:83–91.
[9] Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2.